MEDS Lab Module 4

Section 9: Flip-Flops, Registers & Reset

Task 10: 4-bit Up-Down Counter with Asynchronous Reset

July 23, 2026

1. EDA Playground Link

The complete design and testbench code for the 4-bit Up-Down Counter can be found at the following shareable link:

- `https://edaplayground.com/x/Rea2`

2. Schematic Diagram

Below is the top-level block diagram for the 4-bit Up-Down Counter.

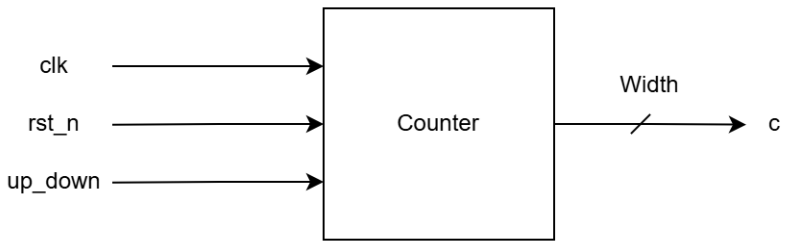

Figure 1: Block diagram of the 4-bit Up-Down Counter.

3. Terminal Output Screenshot

The following screenshot displays the simulation log, confirming the testbench successfully evaluated the design requirements.

```
Compiler version X-2025.06-SP1_Full64; Runtime version X-2025.06-SP1_Full64;  Jul 23 11:18 2026
Verification complete!
$finish called from file "testbench.sv", line 33.
```

Figure 2: Simulation log output from EDA Playground.

4. Waveform Screenshot

The waveform below clearly demonstrates the counter operations, including counting up, counting down, wrap-around behaviors, and asynchronous reset capability.

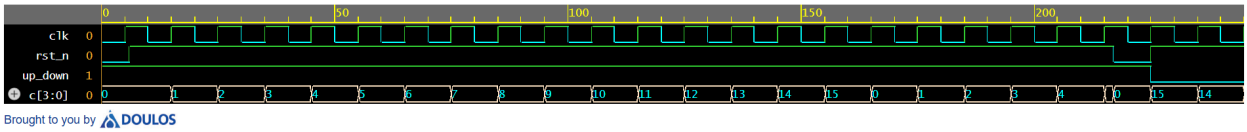

Figure 3: EPWave waveform confirming up-down counting, wrap-around, and asynchronous reset.

5. Explanation

This task involved implementing a 4-bit up-down counter utilizing a parameterizable `always_ff` sequential block. As defined by the module specifications, the counter utilizes an asynchronous active-low reset signal (`rst_n`).

The design relies on the sensitivity list `always_ff @(posedge clk or negedge rst_n)`. This specifies that an update to the counter's state can be triggered either synchronously by the rising edge of the clock or asynchronously by the falling edge of the reset signal.

When `rst_n` drops to 0, the counter state `c` is forced to 0 immediately, irrespective of the clock edge. This immediate reset behavior is visibly verified in the waveform screenshot. Once `rst_n` returns to 1, the counter resumes regular operations, controlled by the `up_down` signal on the rising edge of the clock. It counts up sequentially $(0, 1, 2 \ldots)$ when `up_down = 1` and counts down $(15, 14 \ldots)$ when `up_down = 0`, properly handling overflow and underflow wrap-around limits inherently due to its fixed 4-bit width.